

Probleme d'activation de capteurs pour surveillance de zones

IUT Nord Franche-Comte | Techniques d'optimisation | Karine Deschinkel | 2023-2024

1. Construction des configurations elementaires

Une **configuration elementaire** est un ensemble minimal de capteurs couvrant toutes les zones : aucun capteur ne peut etre retire sans laisser une zone non surveillee. Ces configurations constituent les variables du programme lineaire.

Deux heuristiques ont ete implementees :

Heuristique	Principe	Avantage	Source
Greedy (gloutonne)	A chaque etape, choisit le capteur couvrant le plus de zones non encore couvertes. En cas d'egalite, choix aleatoire.	Configs de bonne qualite individuelle	Cardei & Du (2005)
Aleatoire	Parcourt les zones dans un ordre aleatoire, choisit n'importe quel capteur couvrant la zone courante.	Grande diversite de configurations	Deschinkel (2011)

Dans les deux cas, la configuration est ensuite rendue elementaire en testant la suppression de chaque capteur (ordre aleatoire) : si la suppression maintient la couverture totale, le capteur est retire.

2. Solutions obtenues sur les instances (Partie 4)

Le programme lineaire est resolu avec le solveur CBC (via PuLP, equivalent Python de GLPK). Les resultats suivants sont obtenus avec 10 configurations elementaires generees par l'heuristique greedy (seed=42) :

Instance	N capteurs	M zones	Configs generees	Duree de vie obtenue	Temps (s)	Statut
fichier-exemple	4	3	4	8.5000	0.13	Optimal
moyen_test_2	20	10	3	15.0000	0.17	Optimal
moyen_test_3	10	10	10	358.0000	0.06	Optimal
gros_test_1	100	200	10	177.0000	0.20	Optimal

L'instance fichier-exemple atteint l'optimum prouve (8.5) documente dans le sujet. Pour moyen_test_2, seulement 3 configurations distinctes sont trouvees par le greedy, ce qui limite la qualite de la solution — l'impact du nombre de configurations est analyse en Partie 5.

Probleme d'activation de capteurs — Analyse des resultats

3. Analyse des resultats (Partie 5)

Pour un meme probleme, nous examinons l'influence du **nombre** et du **type** des configurations elementaires sur la duree de vie du reseau.

3.1 Influence du nombre de configurations

Resultats sur `moyen_test_3` (N=10, M=10) avec l'heuristique greedy :

Nb configs	1	2	3	5	10	15	20
Greedy	55.0	127.0	166.0	268.5	358.0	358.0	358.0
Aleatoire	90.0	109.0	163.0	235.0	342.0	395.0	395.0

Tableau 1 — Duree de vie en fonction du nombre de configurations (instance `moyen_test_3`)

Observation : la duree de vie augmente significativement avec le nombre de configurations jusqu'a saturation (plateau a partir de 10-15 configs). Avec 1 seule configuration, on obtient seulement 55.0 contre 395.0 avec 20 configs, soit une amelioration de +618%. Au-dela d'un certain seuil, ajouter des configurations n'apporte plus de gain : toutes les configurations pertinentes ont ete trouvees.

3.2 Influence du type d'heuristique

Instance	Heuristique	Configs reelles	Duree de vie
fichier-exemple	Greedy	4	8.5000
fichier-exemple	Aleatoire	4	8.5000
moyen_test_2	Greedy	3	15.0000
moyen_test_2	Aleatoire	10	58.0000
moyen_test_3	Greedy	10	358.0000
moyen_test_3	Aleatoire	10	342.0000

Tableau 2 — Comparaison des deux heuristiques avec 10 configurations demandees

Observation : aucune heuristique ne domine l'autre systematiquement. Sur `moyen_test_2`, l'aleatoire est nettement superieur (58.0 vs 15.0) car le greedy converge toujours vers les memes capteurs dominants et ne produit que 3 configs distinctes. Sur `moyen_test_3`, le greedy prend l'avantage (358.0 vs 342.0) car ses configs sont individuellement de meilleure qualite. Ces resultats suggerent qu'une **combinaison des deux heuristiques** produirait les meilleurs pools : diversite de l'aleatoire + qualite du greedy.

Conclusion

La qualite de la solution depend directement de la richesse du pool de configurations. Un pool trop petit ou trop homogene bride le programme lineaire. La strategie optimale consiste a generer un nombre suffisant de configurations (≥ 10) en alternant les deux heuristiques pour maximiser la diversite tout en conservant la qualite.